

Deep Learning Aided Sensor Fusion for Drift Reduced IMU Orientation Estimation

Rahul Verma
201629064

Supervisor: Professor. Zhiqiang Zhang
Assessor: Dr. Chris Wood

A project presented for the degree of
Masters of Engineering in Electronic Engineering

School of Electrical and Electronic Engineering
University of Leeds

Abstract

Inertial Measurement Units (IMUs) are widely used in a variety of applications, such as Body Sensor Networks (BSNs) for orientation estimation. However the gyroscope suffers from drift due to sensor bias and noise that when integrated accumulates over time. This project investigates a deep learning-based, approach which aims to mitigate gyroscopic errors which can be integrated with sensor fusion techniques, to achieve more accurate orientation estimates. The proposed deep-learning architecture leverages both neural networks and a temporal history to learn complex and non-linear error patterns in IMU data, exploring if it outperforms a standard Kalman Filter (KF) without learned corrections. The network outputs a correction for the incoming gyroscope sample, and the data used in training, testing and validating the model comes from public datasets, such as the Berlin Robust Orientation Estimation Assessment Dataset (BROAD).

Acknowledgements

I would like to express my sincere gratitude to Professor. Zhiqiang Zhang for his continuous support and guidance throughout this project. Professor. Zhang’s insights and advice during this project, in particular the explanation of quaternion algebra and in-depth explanation of filters, were invaluable in shaping the direction of my work. In addition, offering his personal copy of Body Sensor Networks to aid in my understanding was highly appreciated.

I would also like to acknowledge the authors and maintainers of the publicly available datasets of BROAD and RepoIMU. Their efforts in collecting and labelling high quality IMU data, and optical motion capture ground truth enabled the training, testing, and validation of the neural network. I am grateful for their contributions and cite them accordingly in this report.

Contents

Abstract	1
Acknowledgements	1
List of Figures	2
List of Tables	3
1 Introduction	4
1.1 Inertial Measurement Units and their applications	4
1.2 Problem Statement: IMU Drift and Its Impact	4
1.3 Research Question and Hypotheses	4
2 Societal Impacts	5
3 IMU Basics and Operational Principles	6
3.1 Gyroscope: Operations and Errors	6
3.1.1 Angular Rate Integration	6
3.1.2 Gyroscope Error Sources	6
3.2 Accelerometer: Operations and Errors	7
3.2.1 Accelerometer Error Effects	8
3.3 Magnetometer: Operations and Errors	8
3.3.1 Magnetometer Error Sources and Effects	8
4 Datasets	9
4.1 Dataset Selection	9
4.1.1 BROAD	9
4.1.2 Simulated Data	9
4.1.3 Train, Validation, and Test Dataset Split	9
4.2 Scope and Limitations of BROAD	9
4.3 Data Normalisation	10
4.3.1 Z-Score Normalisation	10
4.3.2 Min/Max Scaling	10
4.3.3 Summary and Choice	10
4.4 Noise Augmentation	10
5 Manifold Unscented Kalman Filter	11
5.1 Kalman Filter Overview	11
5.2 Unscented Kalman Filter	11
5.3 The Manifold Problem	11
5.4 Filter Design	12
5.4.1 State Representation	12
5.4.2 Process Model	12
5.4.3 Measurement Models	12
6 Deep Learning Architecture	13
6.1 Aims of the Deep Learning Model	13
6.2 Model Selection	13
6.2.1 Recurrent Neural Networks (RNNs)	13
6.2.2 Convolutional Neural Networks (CNNs)	14
6.2.3 Temporal Convolutional Networks (TCNs)	15
6.2.4 Summary and Choice	16
6.3 TCN Design	16
6.3.1 Normalisation	16
6.3.2 Activation Functions	17
6.3.3 Regularisation	17
6.3.4 Residual Connections/1x1 Convolution	18

List of Figures

1	Bias Effect on Gyro [7]	7
2	Effect of ARW on Gyro Output [8]	7
3	RNN structure [27]	13
4	1D CNN structure [33]	14
5	CNN structure with dilations [35]	15
6	Structure of residual block	16

List of Tables

1	Key characteristics of the BROAD dataset [14]	9
2	Kalman Filter Terms and Definitions	11
3	MUKF configuration	12
4	Common normalisation methods.	17
5	Common activation functions.	17

1 Introduction

1.1 Inertial Measurement Units and their applications

IMUs are composed of multiple sensors which include a gyroscope, accelerometer, and occasionally a magnetometer. These three sensors measure the angular rate, specific force, and the local magnetic field vector respectively. These measurements can be used to estimate the orientation of an object through integration of the angular rate. This data acquisition is essential for several applications such as BSNs[1], robotics, and autonomous vehicles where these systems rely on high-rate orientation updates.

IMUs have emerged as a key technology due to their ability to work in a self-contained environment. In environments where external references of orientation are unavailable or unreliable, technologies such as IMUs are attractive. Filters, such as the KF, allow the use of accelerometers and magnetometers to guide and correct state estimation, but these also suffer from failures like magnetic disturbances, and high linear accelerations. Due to these limitations, there is significant motivation to explore deep-learning methods that compensate for these conditions and errors.

1.2 Problem Statement: IMU Drift and Its Impact

IMUs have shown promise in determining the orientation of an object in motion. However, IMUs suffer from a limitation called drift. IMU drift is characterised by the accumulation of errors through the integration of the angular rate. The sources of these errors include constant bias, scale factor errors, and others expanded in section 2.1.2. Errors are not exclusive to the gyroscope but also affect the accelerometer and magnetometer. Drift is also dependent on the type of IMU that is used. Lower cost/grade IMUs suffer from drift at a higher magnitude, which results in orientation inaccuracies much quicker compared to higher cost/grade IMUs. Errors that lead to inaccuracies in the orientation estimation of an object determined by the IMU.

Kianifar et al. explored using IMUs for orientation estimation in a clinical setting. They found that for rotation angles parallel to gravity, drift due to gyroscope bias cannot be compensated by the accelerometer[2]. Even with multiple sensors, it is still challenging to find an accurate orientation estimation. Thus it is important to try and address the orientation problem by addressing gyroscopic drift.

Consequently, this project aims to address gyroscopic drift by using deep-learning methods to learn the complex and non-linear nature of biases and errors.

1.3 Research Question and Hypotheses

The question this project aims to answer is: **"Can Deep Learning be used to learn the drift pattern to get drift free orientation estimation?"** While this is the overarching question, it can be broken down to the following:

- How effectively can deep learning learn the drift experienced?
- Can this model be generalised or will the model only apply to a single dataset?
- What are the advantages or disadvantages of using Deep Learning compared to traditional approaches?

The hypothesis that I state of this project is that by incorporating a deep-learning model, we will be able to see a percentage decrease in the orientation error over a period of time.

2 Societal Impacts

3 IMU Basics and Operational Principles

3.1 Gyroscope: Operations and Errors

The gyroscope is the main component that is used to determine the orientation of the object. It measures the angular rate in its orthogonal axes $\omega^x, \omega^y, \omega^z$ which is used to determine the object's orientation at a discrete time. The orientation is determined through the integration of angular rate. In this chapter, the quaternion representation is selected due to being more computationally efficient than other methods [3] and does not suffer from gimbal lock [4]. Gimbal lock is described as a phenomenon where two of the three rotational axes of an object align causing a loss of one degree of freedom (DOF), since the Euler angle representation becomes singular.

3.1.1 Angular Rate Integration

We can define a quaternion that is represented by the angular rate, where ω is the real angular rate, and $\omega_q(t)$ is the pure quaternion. Note: Both the attitude and updated quaternions should be normalised to ensure quaternions represent rotations. $q_k \leftarrow \frac{q_k}{\|q_k\|}$.

$$\omega_q(t) = [0, \omega_x(t), \omega_y(t), \omega_z(t)] \quad (1)$$

The orientation evolves according to [4]

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_k \quad (2)$$

where Δq_k refers to the Special Orthogonal group 3 (SO(3)) exponential map, which maps rotational vectors to incremental rotation quaternions [4].

$$\Delta q_k = \begin{bmatrix} \frac{\omega}{|\omega|} \sin \frac{|\omega| \Delta t}{2} \\ \cos \frac{|\omega| \Delta t}{2} \end{bmatrix} \quad (3)$$

This was all done by using an ideal ω_k , if we were to model the angular rate from a gyroscope as [5]

$$\omega_{m,k} = \omega_k + b_k + n_k \quad (4)$$

where $\omega_{m,k}$ is the measured angular rate, ω_k is the true angular rate, b_k is the bias term, and n_k is the noise term. The real quaternion orientation update is as follows where Δq_k becomes $\Delta q_{m,k}$.

$$\Delta q_{m,k} = \begin{bmatrix} \frac{\omega_{m,k}}{|\omega_{m,k}|} \sin \frac{|\omega_{m,k}| \Delta t}{2} \\ \cos \frac{|\omega_{m,k}| \Delta t}{2} \end{bmatrix} \quad (5)$$

and the update is now

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_{m,k} \quad (6)$$

This shows that the inclusion of the bias and noise accumulates over samples as we move forward to the next k, q_{k-1} will incorporate the previous error terms. This results in an accumulated orientation error.

For more information about this derivation see Chapter 12[4].

3.1.2 Gyroscope Error Sources

Constant Bias

The bias of a gyroscope is the average output from the gyroscope when it is not undergoing any rotation [6]. This is measured in $^\circ/h$ and can be estimated by taking an average of the output. As this bias is constant, the drift it causes grows linearly with time. However, as discussed further in this section other error sources can make this difficult to determine. Figure 1 shows how constant bias changes the output.

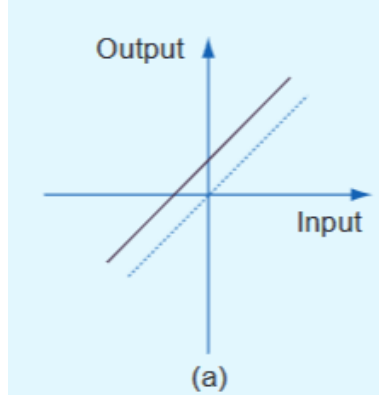


Figure 1: Bias Effect on Gyro [7]

White Noise / Angle Random Walk (ARW)

The gyroscope is also affected by some white noise [6]. This white noise sequence is zero-mean uncorrelated random variables between samples and across axes. When the gyroscope signal is integrated to obtain an angle, this white noise produces an ARW. The units of ARW are denoted by $^{\circ}/\sqrt{h}$, this shows that the deviation of angle error grows proportionally to $\sqrt{t}$. As ARW is due to random variables, it is classified as the 1σ of the orientation error. Analog Devices shows this in figure 2, where the ARW is $0.17^{\circ}/\sqrt{h}$.

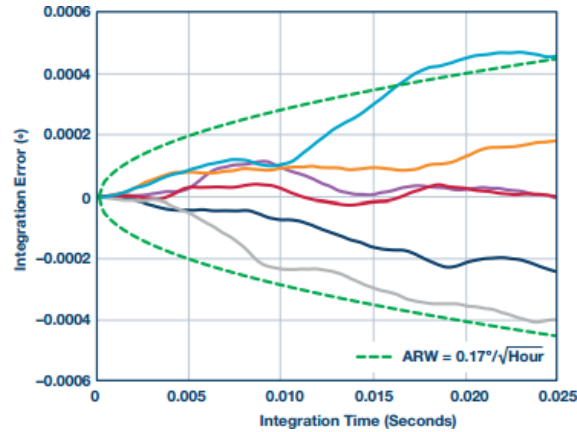


Figure 2: Effect of ARW on Gyro Output [8]

Temperature

Temperature fluctuations due to changes in the environment and sensor heating induce a movement in the bias, this relationship is also highly non-linear [6]. Therefore, it can be very difficult to model and subsequently subtract from the gyroscope measurements compared to a constant bias where temperature is constant.

For more gyroscope error sources and effects, see [6].

3.2 Accelerometer: Operations and Errors

The accelerometer is another component used in determining an orientation estimate of an object. It measures the specific force along its orthogonal axes a^x, a^y, a^z . The main purpose of the accelerometer is to offer a gravity reference to the system such that an accelerometer can provide drift-free inclination estimates [9]. However, the accelerometer, as stated above, does not only measure a gravity reference but also any linear accelerations that the object also experiences. Under conditions of high accelerations, the reference is unreliable and is only reliable for static or slowly moving objects [10]. Additionally, the accelerometer cannot detect rotations about the gravity vector, so it cannot provide a yaw/heading of

the object. However, like the gyroscope, it also suffers from a variety of errors which causes instability in the output.

3.2.1 Accelerometer Error Effects

Accelerometers suffer from very similar sources of errors to the gyroscope (section 2.1.2). Therefore, in this section we will discuss the implications of these errors for our orientation estimates.

Constant Bias

The effect of constant bias is that it causes an offset in the roll/pitch of the system. Unlike constant bias in the gyroscope, it does not grow linearly with time but is a constant offset irrespective of time elapsed.

High Linear Acceleration

As discussed above, the largest problem is high linear accelerations which corrupts the gravity reference. This is a dominant issue in high dynamic motions as the accelerometer will not be able to correct the orientation estimation through fusion, especially if gyroscopic drift is not addressed.

3.3 Magnetometer: Operations and Errors

Magnetometers measure the local magnetic field along its orthogonal axes m^x, m^y, m^z . The magnetometer can be used to determine the heading (yaw) of an object. Sensor fusion/filtering algorithms are dependent on sensing the local magnetic field to eliminate drift in the azimuth (yaw) portion of orientation estimates [11]. It does this by assuming that the measured local field vector provides a reference relative to Earth's field. However, changes in the local field due to other field sources, can distort this measurement which gives rise to erroneous heading corrections.

3.3.1 Magnetometer Error Sources and Effects

Hard-Iron / Soft-Iron

Hard-iron is a constant additive magnetic field applied to the sensor due to the interaction of magnetic fields [12]. This interaction between these sources leads to a constant vector being applied to the measurement.

Soft-iron effects are generated by the interaction of an external magnetic field on any ferromagnetic materials near the sensor [12]. The resulting magnetic field depends on the magnetic field vector with respect to the orientation of the soft iron material [13].

The equations below show how hard-iron and soft-iron effect the magnetic field [12].

$$h_{SI} = C_{SI} R_E^B h^E \quad (7)$$

$$h_{SI+HI} = h_{SI} + b_{HI} \quad (8)$$

where h_{SI} is the soft-iron magnetic field, $C_{SI} \in R^{3 \times 3}$ the soft-iron matrix, $R_E^B \in SO(3)$ the Earth to body rotation matrix, h^E the Earth magnetic field, and b_{HI} the hard-iron bias.

The above discussion on hard-iron/soft-iron effects assumes these disturbances are static relative to the sensor. However, in many real operating conditions, these magnetic fields vary which corrupts the heading reference to a greater degree, the impact of this phenomenon is explored in [FIXME: section](#).

4 Datasets

4.1 Dataset Selection

4.1.1 BROAD

Feature	Description
Sampling Rate	286 Hz
Ground Truth	Optical Motion Capture (OMC) quaternion $[w, x, y, z]$ in ENU reference frame
Number of Trials	39
Number of Samples	1508364 samples
Diverse Trial Set	slow/fast rotations, translations, magnetic disturbances

Table 1: Key characteristics of the BROAD dataset [14]

BROAD [14] is the dataset used in the project. It has several characteristics that make it suitable for selection. First of these is the inclusion of a magnetometer, this project specifically focuses on 9 DOF IMUs, which causes many other datasets to be excluded based on this alone. The OMC ground truth quaternions provide a high accuracy reference which make it a suitable training target for deep learning. The ground truth has also been synchronised with the IMU readings, allowing for supervised learning at every timestep without discontinuities or needing to interpolate.

Additionally, BROAD trials include slow and fast rotations, isolated accelerometer and magnetometer disturbances, and combined scenarios. This means that a variety of trial types across the set can be split such that train, validation, and test sets are equally represented. Hence, the evaluation of the deep learning method reflects generalisation across disturbance conditions. Furthermore, BROAD includes rest phases for each trial, which allows for sensor noise characteristics to be determined empirically. This is expanded upon in [FIXME: sections](#).

4.1.2 Simulated Data

Simulators (MATLAB’s Navigation Toolbox [15]), can be used to generate a diverse IMU dataset with the advantage of near perfect ground truth quaternions. This is due to the ability of creating trials that focus on different scenarios and define the motion profile the IMU undergoes. However, they suffer from major restrictions that do not make them suitable.

The fundamental problem is that the real sensor characteristics are hard to model as these simulations mainly use simple Gaussian processes. Real noise is more complex with bias being temperature dependent, noise is correlated across axes, and magnetic disturbances vary both spatially and temporally. This makes deep learning models trained on this data harder to generalise to real scenarios (sim to real gap) as the data does not capture the same level of complexity in error sources described in section 2.1.2. This domain shift causes a model’s evaluation on real IMUs to underperform.

Moreover, simulation causes leakage into deep learning design decisions as simulated sensor noise profiles are known beforehand. This leads to any decisions made to be circular in nature.

Due to the main restriction of the sim to real gap, the choice was made to use the BROAD dataset.

4.1.3 Train, Validation, and Test Dataset Split

4.2 Scope and Limitations of BROAD

The BROAD dataset is not without its limitations, the main limitation is that it contains a small number of distinct trials, which limits the diversity of conditions the model can be exposed to. In comparison, most conventional deep learning tasks such as natural language processing and image processing tend to have more diverse pool in its training set which allows for their solutions to generalise and converge easier. This limitation can lead to poorer generalisation to our test set and thus may reduce performance on unseen conditions. Hestness et al empirically demonstrates this through a power-law which states that generalisation error decreases as the training set increases [16]. However, adding noise augmentation [FIXME: section](#) can artificially diversify the training set and is able to alleviate this limitation but not fully solve it.

Further, the data collected was in a controlled laboratory setting, where clean and predictable conditions are present. In a real deployment, temperature, and scale factor errors may be more present compared to the dataset. This can hurt generalisation even if the same hardware is being used. However, to try and alleviate this, the model was tested on the more disturbed conditions (see **FIXME: section**) to try and test the model at these extreme conditions.

4.3 Data Normalisation

Data normalisation is the process of converting a variable's actual range to a standard range of values that is seen at a neural network's (NN) input. LeCun et al highlights normalisation by stating that convergence is usually faster if the average of the input variable over training is close to zero [17]. This is due to any shift in the average input from zero will bias NN weight updates in a certain direction be it positive or negative. Meaning, that if the network wanted to shift weights in the opposite direction it must 'zigzag' which is inefficient and slow [17]. Sola et al expands on normalisation stating we can expect better predictions from our NN by utilising normalisation. It was concluded that with adequate normalisation of input data, estimation errors can be reduced by a factor of by a factor between 5-10 [18].

Thus, data normalisation was considered to be a very important addition to the pre-processing pipeline.

4.3.1 Z-Score Normalisation

4.3.2 Min/Max Scaling

4.3.3 Summary and Choice

4.4 Noise Augmentation

5 Manifold Unscented Kalman Filter

5.1 Kalman Filter Overview

The Kalman filter (KF) is an optimal recursive estimator used to estimate a state under noisy measurements and uncertain dynamics. It works under an assumption that the relationship between our state and measurements can be modelled as linear and Gaussian [19].

Term	Description
State Vector ($\mathbf{x}$)	The quantity the filter is trying to estimate.
State Estimate Covariance ($\mathbf{P}$)	The uncertainty the filter has in its state estimation.
Process Noise Covariance ($\mathbf{Q}$)	The uncertainty the filter has in its process model, i.e. how much the filter trusts the dynamics used to propagate the state.
Measurement Noise Covariance ($\mathbf{R}$)	The uncertainty the filter has in its measurement values, used to correct the state.
Kalman Gain ($\mathbf{K}$)	Balances the weight given to the process model prediction versus the incoming measurement.

Table 2: Kalman Filter Terms and Definitions

The filter implements a two step process of prediction and update. The prediction propagates our state estimation and the update corrects the state estimation using one or multiple measurements. The covariance $\mathbf{P}$ is updated by the $\mathbf{K}$ after every new sensor value to reflect the new information the filter has gained about the system.

However, the KF breaks down for orientation estimation due to some key factors. First, quaternion kinematics are non-linear (**FIXME: see SO(3) equation**) and the assumption that the relationship is linear is violated. Second, as the relationship is non-linear, this breaks the Gaussian noise process model as propagation through a non-linear function produces a non-Gaussian output. Third, as quaternions exist in SO(3) and not Euclidean space, adding a Gaussian term to the state can push it off the manifold [20].

5.2 Unscented Kalman Filter

The Unscented Kalman Filter (UKF) is an extension of the KF that allows for non-linear relationships to be more accurately modelled by the filter. Traditionally, to apply a KF to a non-linear relationship an extended KF (EKF) is used which linearises all non-linear models through the use of Jacobian matrices that are evaluated at the current state estimate [21]. However, better performance tends to be achieved by a UKF for orientation estimation as a UKF gives more accurate covariance propagation at larger rotations. This is due to UKFs are able to capture second order processes through the use of sigma points [22] [21].

Sigma points are drawn from a specific, deterministic algorithm that when a non-linear function is applied to these points; a set of $2n+1$ transformed points are left. These sigma points are then used to reconstruct the mean by taking a weighted average of these points and the weighted outer product for the covariance.

However, with the non-linear propagation addressed, computing a weighted mean of quaternions directly is geometrically invalid. This is due to quaternions existing on the SO(3) manifold, as the element-wise average of quaternions does not lie on the unit sphere and therefore does not represent a valid rotation.

For an in-depth derivation regarding the UKF see [21].

5.3 The Manifold Problem

The core issue of the UKF is that it assumes that the $\mathbf{x}$ lies within Euclidean space, however quaternions lie on the SO(3) manifold. So computing the mean and adding the measurement noise to the quaternion state is geometrically invalid, thus the state that is estimated is also invalid.

The solution to this problem is to perform any arithmetic computations that the UKF requires within the local tangent space (so(3)). By taking the current state estimate (quaternion on the SO(3) manifold) and mapping it to and from Euclidean vector space, the predict and update steps are performed validly

and the geometric representation of the state is conserved. Brossard et al and Crassidis et al explore and show this within their UKF-Manifold and Unscented Quaternion Estimator (USQUE) implementations [23] [24].

Thus the implementation used in this project is based on UKF-M as it is generalisable to various manifolds not just $SO(3)$.

5.4 Filter Design

Table 3 below shows the configuration used for the MUKF.

Parameter	Value	Description
State	$\mathbf{q} \in SO(3)$	Unit quaternion $[w, x, y, z]^T$
Error state	$\delta\phi \in R^3$	Rotation vector in tangent space $so(3)$
Sigma points	$2n + 1 = 7$	$n = 3$, spread parameter $\kappa = 1$
Process noise $\mathbf{Q}$	$\text{diag}(\sigma_\omega^2) \cdot \Delta t$	σ_ω from BROAD rest-phase measurements
Accel. noise $\mathbf{R}_{\text{acc}}$	$\text{diag}([0.044, 0.050, 0.074]^2) \text{ m}^2/\text{s}^4$	From BROAD rest-phase measurements
Mag. noise $\mathbf{R}_{\text{mag}}$	$\text{diag}([0.71, 0.70, 0.68]^2) \mu\text{T}^2$	From BROAD rest-phase measurements
Accel. gate	5% of $\ \mathbf{g}\ $	Rejects updates under dynamic acceleration
Mag. gate	30% of $\ \mathbf{m}_{\text{ref}}\ $	Rejects updates under magnetic disturbance
Sampling rate	286 Hz	Matches BROAD dataset

Table 3: MUKF configuration

5.4.1 State Representation

The state represents the quaternion in $SO(3)$ and the error state is the uncertainty the MUKF has in its orientation estimate. This is a significant deviation compared to the interim implementation. This is because the deep-learning model is used to correct the gyroscope signal, meaning that if the filter also tried to estimate this error in the gyroscope signal there would be a conflict between the filter and model. The 3D error state lives in local tangent space ($so(3)$), this is so the predict and update steps that need to happen in Euclidean space can be performed correctly and then mapped to $SO(3)$ using **FIXME: equation**.

5.4.2 Process Model

First, sigma points are generated from the current state estimate. For our case $n = 3$ for a 3D error state, giving 7 sigma points. Then each sigma point is propagated through our process model, which is the **FIXME: equation**. This is the predict step of the filter and gives us the next predicted state of the filter (quaternion estimate). After propagating the sigma points, the predicted mean is recovered via the Fréchet mean [23], and the predicted covariance is reconstructed from the weighted outer product of the tangent space errors (3D errors).

5.4.3 Measurement Models

The measurement models are the accelerometer and magnetometer that are used to update and correct the state estimate from the prediction. The accelerometer update compares the measured specific force against the gravity vector (ENU frame) rotated into body frame. As mentioned in **FIXME: section**, this corrects the tilt (roll/pitch) of the state estimate. The magnetometer is compared against the horizontal magnetic reference which is collected over the first 50 samples which is in the rest phase [14]. This is so the filter has a clean magnetic reference before any disturbances occur. Again as mentioned in **FIXME: section** this corrects the azimuth/yaw of the current filter estimate.

6 Deep Learning Architecture

6.1 Aims of the Deep Learning Model

The motivation for learning $\Delta\omega_t = [\Delta\omega_x, \Delta\omega_y, \Delta\omega_z]^\top$ comes from the fact that while sensor fusion techniques of downweighting or skipping of the accelerometer and magnetometer when they are unreliable, drift cannot be mitigated due to the accumulation of gyroscopic bias and noise [25][26].

6.2 Model Selection

6.2.1 Recurrent Neural Networks (RNNs)

The first type of model that was considered is a RNN. RNNs stem from an idea of recurrence, that is that the output depends on some input at time t and an internal state of the model that holds information about previous outputs of the model. The general idea can be described by the equation

$$\hat{y} = f_{model}(x_t, h_{t-1}) \quad (9)$$

where $\hat{y}$ is the output, f_{model} , is a function that describes the model, x_t is the input at time t , and h_{t-1} is a vector state that describes the internal state of the model at the previous timestep. Expanding on this, the state of the model is updated as follows

$$h_t = f_w(x_t, h_{t-1}) \quad (10)$$

where f_w is a function with weights w . This expands to the equations

$$\hat{y} = W_{hy}^T h_t \quad (11)$$

$$h_t = \phi(W_{hh}^T h_{t-1} + W_{xh}^T x_t) \quad (12)$$

where $W_{hh}^T, W_{hy}^T, W_{xh}^T$ are a set of weight matrices that are reused for every timestep, and ϕ represents an arbitrary activation function.

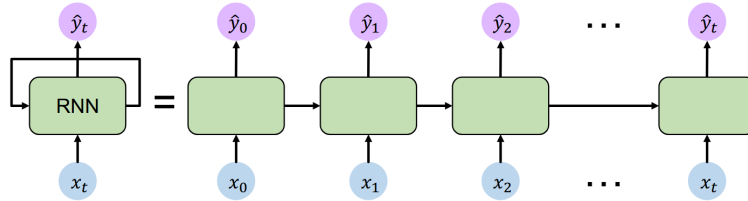


Figure 3: RNN structure [27]

Figure 7 shows the unrolled flow of the RNN, where the arrows between each block represents the passing of h_t , from one input to the next. The point of the recurrence state h_t , is to show how the output of the model depends on context from previous inputs plus h_t . This is especially relevant in our case, as described in section 2.1.1, the accumulation of gyroscopic errors comes from the integration of the angular rate. As these errors accumulate, the current error depends on the history of the gyroscope measurements.

There are many problems with RNNs however, the biggest being the problem of exploding/vanishing gradients. RNNs are trained via backpropagation through time, which propagates gradients across timesteps. For long sequences, computing the gradient with respect to h_0 involves many factors of repeated multiplication of weight matrices across time. This can lead to many values of the computation being greater than 1, which explodes gradients leading to unstable training. In the case, where many values are less than 1, this causes vanishing gradients where learning long-range dependencies becomes difficult. Additionally,

as the hidden state h_t is propagated sequentially through time, when performing backpropagation, training cannot be parallelised over timesteps. This results in high training times compared to convolutional models.

Updated versions of the RNN have been proposed to solve the problem of vanishing and exploding gradients. These being Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs). These methods exploit the use of a cell state that does not need to perform repeated multiplication of weight matrices, however, they are still subject to long training times and the gradient problem is not fully solved but mitigated [28][29].

6.2.2 Convolutional Neural Networks (CNNs)

The next type of model considered was the CNN, even though typically it is used in computer vision tasks; it has seen wide use in research relating to drift reduction [30][31]. This model is based on the idea of convolving input data with a learnable filter to extract local features in the input data. The filter can be thought of like a window that slides over the data, and each filter has a defined set of weights in which it multiplies with input data. This convolution operation can be described by the equation below:

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t + k - \lfloor K/2 \rfloor] \quad (13)$$

$$y_f[t] = \phi(z_f[t]) \quad (14)$$

where $y_f[t]$, is the output of filter f , b_f is a bias, C_{in} is the input channels (for our case C_{in} would be 3 for $\omega_x, \omega_y, \omega_z$), K is the kernel/filter width, $w_{f,c}$ is the weights of the filter for a specific channel, x_c is the input data of that channel, and ϕ is an arbitrary non-linear activation function (typically ReLU).

For example if $K = 3$, the convolution will occur over the timesteps $x[t-1], x[t], x[t+1]$, this is how the convolution extracts local features in the input data.

Typically, these CNNs are made up of layers, where in each layer the number of filters grows e.g. $L1 = 16, L2 = 32, L3 = 64$ etc. These additional filters in later layers allow the model to learn more complex feature sets based on the previous layers' filter. Zeiler et al. [32] shows this in their figure 2 where their convolutional network is used for image classification.

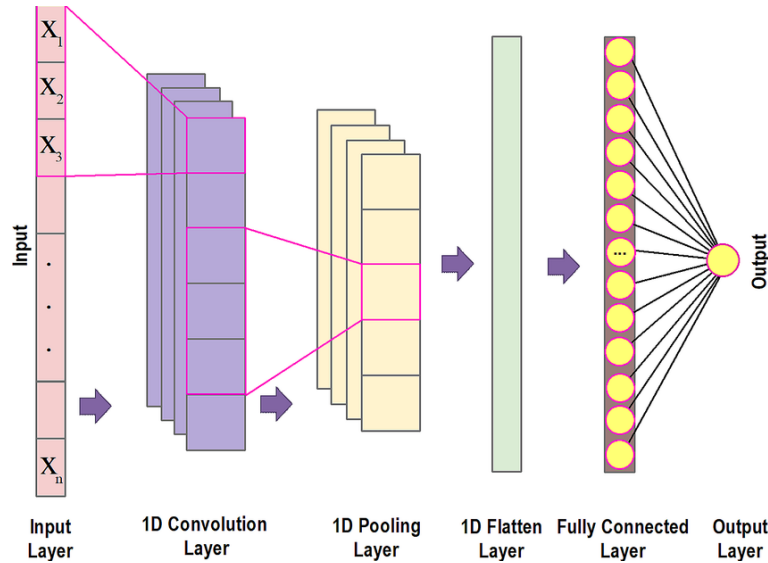


Figure 4: 1D CNN structure [33]

CNNs are more attractive than RNNs due to its ability to be efficiently trained by using parallelisation [34]. This leads to the ability to test multiple different iterations of a model where parameters

and loss functions may be changed to explore what effects they may have. CNNs also do not suffer from vanishing/exploding gradients as much and are seen to be more stable during training due to no backpropagation through time.

However, unlike RNNs, baseline CNNs cannot produce outputs due to significant temporal information. It can extract local features and in 1D convolutions it has some ability to traverse backwards through time but not to any significant degree. As seen in section 2.1.1, the accumulation of drift is due to the integration of the angular rate as we move forward in time.

6.2.3 Temporal Convolutional Networks (TCNs)

TCNs are an extension of the baseline CNN and include two main differences in their implementation as shown by Bai et al.[35]. These differences are the use of dilations that grow the receptive field allowing the network to look back further in time and learn long-term information. Furthermore, causal convolutions meaning, that the convolution operation cannot look forward in time. To explain causality, we can look back at equation 13 and modify this so

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t - k] \quad (15)$$

$$y_f[t] = \phi(z_f[t]) \quad (16)$$

The difference is that the k is subtracted from $x[t]$. Causality is enforced here as at time t , we cannot access future information i.e. $x[t + 1], x[t + 2] \dots$ etc.

Dilations allow the model to look further back in time without the need of increasing parameters for the same K and C_{in} used in convolutions. This is best represented in figure 9.

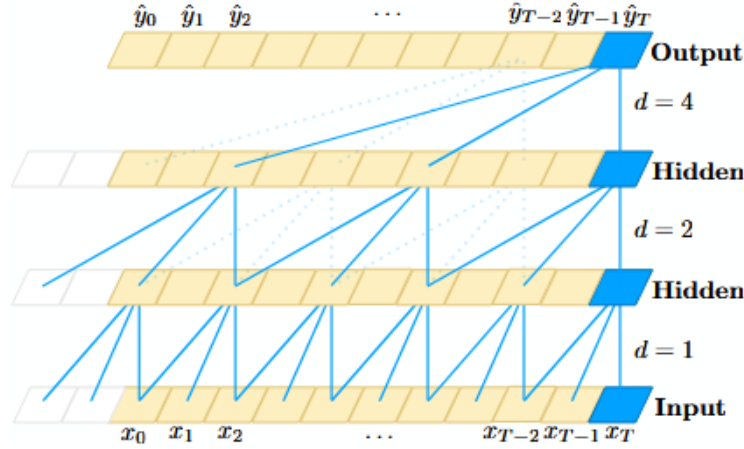


Figure 5: CNN structure with dilations [35]

As we can see in the diagram, as the dilations increase layer by layer, the output to the next layer looks further back into the history of $x[t]$. This can be modelled by the equation

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t - dk] \quad (17)$$

where d is the dilation factor of the layer.

6.2.4 Summary and Choice

In summary, the choice has been to select the TCN. This is because it can look back in history and model long-term dependencies between data, something baseline CNNs cannot do. Additionally, like a CNN it is quicker and more stable in training unlike an RNN. However, the TCN does have its problems, these being extra data storage during its operation and domain transfer. Unlike RNNs which only have store $h(t)$, TCNs need to take in a raw sequence up to its effective history length [35]. Domain transfer refers to transferring a model that has been trained in one domain (i.e. small receptive field) and applied to another domain (i.e. large receptive field), where the TCN may perform poorly due to this change in domain.

6.3 TCN Design

The main design of the TCN comes from Bai et al. [35]. Figure 10 demonstrates the architecture of the residual block that has been implemented in the first semester.

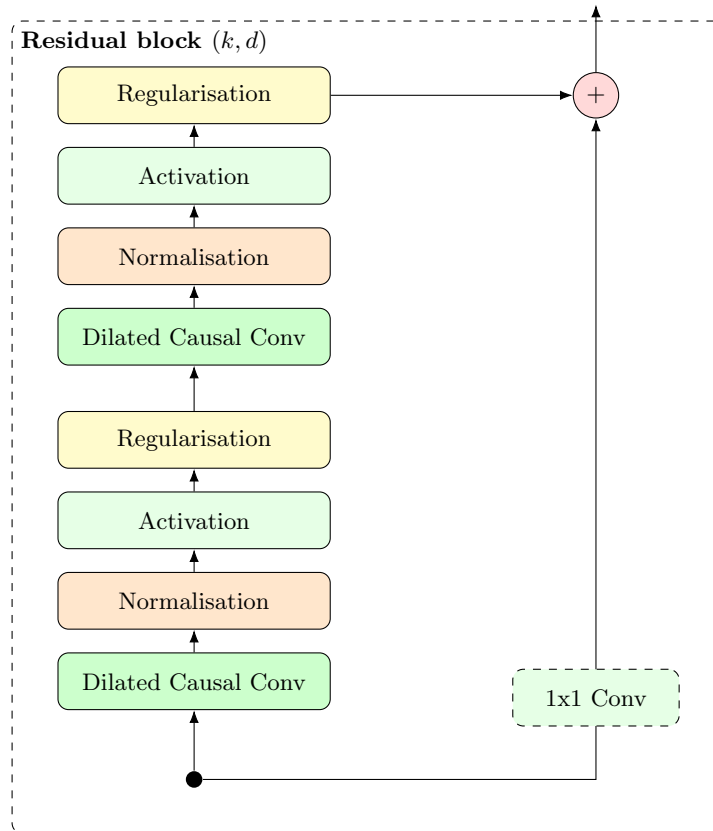


Figure 6: Structure of residual block

The dilated causal convolution is the same as discussed in section 4.2.3.

6.3.1 Normalisation

Normalisation is a technique used in deep learning to ensure training stability, while also reducing training times by allowing the use of higher learning rates. The aim of normalisation is to address internal covariate shift, the change in the distribution of each layer's inputs during training, as parameters of the previous layers change [36]. This can be done by the following techniques.

Method	Normalises	Pros	Cons
BatchNorm	Activations (per channel)	Fast and adds some regularisation	small batch/streaming issues
LayerNorm	Activations (per token/sample)	Batch-size independent	sometimes worse performance in CNNs
GroupNorm	Activations (channel groups)	Works well with small batches	Number of groups need to be selected
WeightNorm	Weights (reparam)	Simple no batches	less regularisation than BatchNorm

Table 4: Common normalisation methods.

From the table above, in training BatchNorm, GroupNorm, and WeightNorm will be tested to see which technique yields the best model accuracy to model training stability. The reason why these have been selected is that BatchNorm has seen wide use in CNN models for drift reduction [30][31]. WeightNorm as the original TCN implementation uses it to avoid errors with small batch sizes [35]. GroupNorm as it is a middle ground where it works well with small batch sizes compared to BatchNorm but it may introduce more complexity with groups needing to be selected and as such will be tested last to see if the other techniques give an acceptable trade-off.

6.3.2 Activation Functions

Activation functions are functions that are used in deep learning to add non-linearity to our output. This is because seen in equations 11,12,13,15,17 the relationship between weights and input data is all linear. The table below demonstrates common activation functions used.

Activation	Definition	Output range	When used	Pros	Cons
ReLU	$\text{ReLU}(x) = \max(0, x)$	$[0, \infty)$	CNN default baseline	Fast activations	ReLU can result in 0 output (dying ReLU)
Leaky ReLU	$\max(\alpha x, x), \alpha \in (0, 1)$	$(-\infty, \infty)$	When units die	Reduces dying ReLU	extra hyperparameter α
GELU	$x \Phi(x)$ (Gaussian CDF)	$(-\infty, \infty)$	Transformers, deep networks	Smooth and good performance	slower than ReLU
Swish	$x \sigma(\beta x)$	$(-\infty, \infty)$	Transformers /CNNs	Smooth, often improves accuracy	slower than ReLU

Table 5: Common activation functions.

From the table above, the motivation for ReLU is that it is commonly used as the baseline for CNNs and is also used by Bai et al. in their TCN implementation. However, they can be prone to overfitting [30] or dying ReLU can occur. If ReLU is prone to dying ReLU, then leaky ReLU will be used and tested against the other functions. GELU and Swish are known to be smooth implementations of ReLU, where it is able to take values where $x < 0$ and can have slight performance improvements [37] but are computationally more expensive and performance is model dependent. For implementation, ReLU will be considered first for its computational efficiency, however if problems arise the other activation functions will be used.

6.3.3 Regularisation

Regularisation is a technique used in deep learning models to help models avoid overfitting to training data and allows the model to be more generalisable to unseen data. Overfitting is when the model learns its training data too precisely, so it performs well to training data but less than adequate to unseen data.

The technique that will be implemented first is dropout, where dropout randomly zeroes out all activations within the model for an input in training data. If the model is still prone to overfitting, more advanced regularisation techniques may be employed like weight decay which penalises large weights within kernels to allow for smoother functions.

6.3.4 Residual Connections/1x1 Convolution

The right branch (figure 10) is essential in training very deep neural networks where networks may not be able to converge due to exploding/vanishing gradients. This branch is called a residual connection, where the equation below shows its operation

$$y = \mathcal{F}(x, W_i) + Wx \quad (18)$$

where y is the output, $\mathcal{F}(x, W_i)$ is the residual mapping to be learned, W , is a projection applied to x (1x1 convolution), and x is our input[38].

Residual connections add inputs of a block to the output of the block, so when learning the model learns the relationship of $\mathcal{F}(x, W_i) = y - Wx$, which is easier to learn than the full mapping directly. The 1x1 convolution is included as the model goes deeper, the number of channels increases to capture more complex features as each channel has their own set of filters/kernels. The addition between $\mathcal{F}(x, W_i)$ and x must be of two tensors of the same size so W allows the addition to be possible.

References

- [1] A. Filippeschi, N. Schmitz, M. Miezal, G. Bleser, E. Ruffaldi, and D. Stricker, “Survey of motion tracking methods based on inertial sensors: A focus on upper limb human motion,” *Sensors*, vol. 17, p. 1257, Jun. 2017. DOI: 10.3390/s17061257.
- [2] R. Kianifar, V. Joukov, A. Lee, S. Raina, and D. Kulić, “Inertial measurement unit-based pose estimation: Analyzing and reducing sensitivity to sensor placement and body measures,” *Journal of Rehabilitation and Assistive Technologies Engineering*, vol. 6, p. 205 566 831 881 345, Jan. 2019. DOI: 10.1177/2055668318813455. [Online]. Available: <https://journals.sagepub.com/doi/full/10.1177/2055668318813455>.
- [3] Z.-Q. Zhang and J.-K. Wu, “A novel hierarchical information fusion method for three-dimensional upper limb motion estimation,” *IEEE Transactions on Instrumentation and Measurement*, vol. 60, pp. 3709–3719, Nov. 2011. DOI: 10.1109/tim.2011.2135070. (visited on 11/27/2021).
- [4] G.-Z. Yang, *Body Sensor Networks*. London Springer, 2014.
- [5] N. Trawny and S. Roumeliotis, *Indirect kalman filter for 3d attitude estimation*, 2005. [Online]. Available: <https://mars.cs.umn.edu/tr/reports/Trawny05b.pdf> (visited on 01/09/2026).
- [6] O. J. Woodman, “Number 696 an introduction to inertial navigation,” *An introduction to inertial navigation*, 2007. [Online]. Available: <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-696.pdf>.
- [7] “How good is your gyro [ask the experts],” *IEEE Control Systems*, vol. 30, pp. 12–86, Feb. 2010. DOI: 10.1109/mcs.2009.935122. (visited on 08/31/2021).
- [8] M. Looney, *Designing for low noise feedback control with mems gyroscopes*, Analog.com, Mar. 2016. [Online]. Available: <https://www.analog.com/en/resources/technical-articles/low-noise-feedback-control-mems-gyroscopes.html> (visited on 01/09/2026).
- [9] A. M. Sabatini, “Estimating three-dimensional orientation of human body parts by inertial/magnetic sensing,” *Sensors*, vol. 11, pp. 1489–1525, Jan. 2011. DOI: 10.3390/s110201489.
- [10] *Detection of static and dynamic activities using uniaxial accelerometers — ieee journals & magazine — ieee xplore*, ieeexplore.ieee.org. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=547939>.
- [11] E. Bachmann, X. Yun, and C. W. Peterson, “An investigation of the effects of magnetic variations on inertial/magnetic orientation sensors,” Jan. 2004. DOI: 10.1109/robot.2004.1307974. (visited on 08/02/2023).
- [12] J. Luiz, G. H. Elkaim, C. Silvestre, P. Oliveira, and B. Cardeira, “Geometric approach to strapdown magnetometer calibration in sensor frame,” vol. 47, pp. 1293–1306, Apr. 2011. DOI: 10.1109/taes.2011.5751259.
- [13] *Calibration of a magnetometer in combination with inertial sensors*, [Ieee.org](http://ieeexplore.ieee.org), 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/6289882> (visited on 01/10/2026).
- [14] D. Laidig, M. Caruso, A. Cereatti, and T. Seel, “Broad—a benchmark for robust inertial orientation estimation,” *Data*, vol. 6, p. 72, Jun. 2021. DOI: 10.3390/data6070072.
- [15] MathWorks, *Navigation toolbox*, Accessed: 2026-05-03, 2026. [Online]. Available: <https://uk.mathworks.com/products/navigation.html>.
- [16] J. Hestness, S. Narang, N. Ardalani, *et al.*, *Deep learning scaling is predictable, empirically*. [Online]. Available: <https://arxiv.org/pdf/1712.00409>.
- [17] Y. LeCun, L. Bottou, G. B. Orr, and K. -r. Müller, “Efficient backprop,” *Lecture Notes in Computer Science*, pp. 9–50, 1998. DOI: 10.1007/3-540-49430-8_2. [Online]. Available: https://link.springer.com/chapter/10.1007%2F3-540-49430-8_2.
- [18] J. Sola and J. Sevilla, “Importance of input data normalization for the application of neural networks to complex industrial problems,” *IEEE Transactions on Nuclear Science*, vol. 44, pp. 1464–1468, Jun. 1997. DOI: 10.1109/23.589532.
- [19] G. Welch and G. Bishop, *An introduction to the kalman filter*, 2006. [Online]. Available: https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf.
- [20] E. Gallo, *The so(3) and se(3) lie algebras of rigid body rotations and motions and their application to discrete integration, gradient descent optimization, and state estimation*, arXiv.org, 2022. [Online]. Available: <https://arxiv.org/abs/2205.12572> (visited on 05/12/2026).

- [21] S. Julier and J. Uhlmann, *A new extension of the kalman filter to nonlinear systems*, 1997. [Online]. Available: https://www.cs.unc.edu/~welch/kalman/media/pdf/Julier1997_SPIE_KF.pdf.
- [22] E. Kraft, “A quaternion-based unscented kalman filter for orientation tracking,” Jan. 2003. DOI: 10.1109/icif.2003.177425.
- [23] M. Brossard, A. Barrau, and S. Bonnabel, “A code for unscented kalman filtering on manifolds (ukf-m),” *HAL (Le Centre pour la Communication Scientifique Directe)*, May 2020. DOI: 10.1109/icra40945.2020.9197489. (visited on 04/17/2024).
- [24] J. Crassidis and F. Landis Markley, *Unscented filtering for spacecraft attitude estimation*. [Online]. Available: https://www.acsu.buffalo.edu/~johnnc/uf_att.pdf (visited on 05/12/2026).
- [25] Z.-Q. Zhang, X.-L. Meng, and J.-K. Wu, “Quaternion-based kalman filter with vector selection for accurate orientation tracking,” *IEEE Transactions on Instrumentation and Measurement*, vol. 61, pp. 2817–2824, Oct. 2012. DOI: 10.1109/tim.2012.2196397. (visited on 09/05/2020).
- [26] F. Bo, J. Li, W. Wang, and K. Zhou, “Robust attitude and heading estimation under dynamic motion and magnetic disturbance,” *Micromachines*, vol. 14, pp. 1070–1070, May 2023. DOI: 10.3390/mi14051070. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10221362/>.
- [27] A. Soleimany, *Deep sequence modeling mit 6.s191*, 2019. [Online]. Available: https://introtodeeplearning.com/2019/materials/2019_6S191_L2.pdf (visited on 01/15/2026).
- [28] S. Kanai, Y. Fujiwara, and S. Iwamura, *Preventing gradient explosions in gated recurrent units*. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/file/f2fc990265c712c49d51a18a32b39f00/Paper.pdf (visited on 01/15/2026).
- [29] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” *ResearchGate*, Nov. 2012. [Online]. Available: https://www.researchgate.net/publication/233730646_On_the_difficulty_of_training_Recurrent_Neural_Networks.
- [30] M. Brossard, S. Bonnabel, and A. Barrau, “Denoising imu gyroscopes with deep learning for open-loop attitude estimation,” *IEEE Robotics and Automation Letters*, pp. 1–1, 2020. DOI: 10.1109/lra.2020.3003256. (visited on 06/29/2022).
- [31] H. Chen, P. Aggarwal, T. M. Taha, and V. P. Chodavarapu, *Improving inertial sensor by reducing errors using deep learning methodology*, IEEE Xplore, Jul. 2018. DOI: 10.1109/NAECON.2018.8556718. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8556718>.
- [32] M. Zeiler and R. Fergus, *Visualizing and understanding convolutional networks*, Nov. 2013. [Online]. Available: <https://arxiv.org/pdf/1311.2901>.
- [33] P. V. Hoa, N. A. Binh, P. V. Hong, *et al.*, “One-dimensional deep learning driven geospatial analysis for flash flood susceptibility mapping: A case study in north central vietnam,” *Earth Science Informatics*, vol. 17, pp. 4419–4440, Jul. 2024. DOI: 10.1007/s12145-024-01285-8. (visited on 06/12/2025).
- [34] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, pp. 84–90, May 2012.
- [35] S. Bai, J. Zico Kolter, and V. Koltun, *An empirical evaluation of generic convolutional and recurrent networks for sequence modeling*, Apr. 2018. [Online]. Available: <https://arxiv.org/pdf/1803.01271>.
- [36] S. Ioffe and C. Szegedy, *Batch normalization: Accelerating deep network training by reducing internal covariate shift*, 2015. [Online]. Available: <https://proceedings.mlr.press/v37/ioffe15.pdf>.
- [37] P. Ramachandran, B. Zoph, and Q. V. Le, *Searching for activation functions*, arXiv.org, 2017. [Online]. Available: <https://arxiv.org/abs/1710.05941>.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, Jun. 2016. DOI: 10.1109/cvpr.2016.90. [Online]. Available: <https://ieeexplore.ieee.org/document/7780459>.